অধ্যায় ৩০

JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি ব্যবসার প্রয়োজন থেকে Functional ও Non-functional Requirement, API চুক্তি এবং বাস্তবায়নের পরিকল্পনা তৈরি করতে।
- Frontend, Backend, Database ও নিরাপত্তা একসঙ্গে ভেবে একটি পূর্ণাঙ্গ Application তৈরি করতে।
- Trust Boundary-তে Input যাচাই, নিরাপদ Password ও Session, Server-নির্ধারিত দাম, Idempotency এবং Transaction-ভিত্তিক Inventory বাস্তবায়ন করতে।
- প্রতিটি Requirement-কে Code ও Test-এর প্রমাণের সঙ্গে যুক্ত করতে (Traceability)।
- Unit, Integration, Concurrency, Browser End-to-End Test এবং Type Check দিয়ে একটি Quality Gate সাজাতে।
- Architecture Decision Record (ADR) লিখতে এবং Production-এর জন্য একটি সৎ উন্নয়ন-পরিকল্পনা তৈরি করতে।
- ১০০ নম্বরের মানদণ্ডে নিজের কাজ মূল্যায়ন করতে।
- ১২ সপ্তাহের একটি পেশাদার Roadmap, Portfolio ও Interview-এর প্রস্তুতি পরিকল্পনা করতে।
- **TravelMaster Pro** নামে একটি পূর্ণাঙ্গ, পরীক্ষিত Full-Stack Capstone তৈরি করতে।

পূর্বশর্ত

এই অধ্যায় পুরো বইয়ের ওপর দাঁড়িয়ে। বিশেষভাবে কাজে লাগবে:

- অধ্যায় ১৭: Form ও DOM Event; অধ্যায় ২০: Authentication, Password Hash, Security Header।
- অধ্যায় ২২: Test-এর স্তর; অধ্যায় ২৪: TypeScript ও JSDoc Type।
- অধ্যায় ২৫: HTTP API-র নকশা ও Error চুক্তি; অধ্যায় ২৬: SQLite, Constraint ও Transaction।
- অধ্যায় ২৮: Deployment ও Monitoring; অধ্যায় ২৯: Idempotency ও Consistency।

এই অধ্যায় কেন গুরুত্বপূর্ণ

উনত্রিশটি অধ্যায়ে আপনি আলাদা আলাদা দক্ষতা শিখেছেন: Variable থেকে Closure, DOM থেকে Event Loop, HTTP থেকে Database Transaction, Test থেকে Deployment, Real-Time থেকে Saga। কিন্তু চাকরির প্রথম দিনে কেউ বলবে না "একটি Closure লিখুন"। বলবে: "গ্রাহকরা Package Book করতে পারবেন—দুবার ক্লিক করলে দুটি Booking যেন না হয়, শেষ আসন যেন দুজনকে বিক্রি না হয়, আর কেউ যেন অন্যের Booking বাতিল করতে না পারে।"

এই অধ্যায় সেই কাজটিই করে। একটি সম্পূর্ণ Application—**TravelMaster Pro**—যেখানে প্রতিটি Requirement একটি Code-এর অংশে এবং একটি Test-এ প্রমাণিত। আর শেষে একটি প্রশ্ন: এরপর কী? কীভাবে এই দক্ষতাকে একটি পেশায় রূপ দেবেন।

৩০.১ JavaScript Mastery মানে কী

Syntax মুখস্থ রাখা Mastery নয়। Mastery হলো Correctness, রক্ষণযোগ্যতা, নিরাপত্তা, পর্যবেক্ষণযোগ্যতা, Performance, Accessibility ও পণ্য-ভাবনা—এগুলো মিলিয়ে একটি বাস্তব সমস্যার সমাধান করা, এবং কেন এভাবে করলেন তা ব্যাখ্যা করতে পারা। একজন পেশাদার Developer প্রতিটি সিদ্ধান্তে জিজ্ঞাসা করেন: এটি ব্যর্থ হলে কী হবে? কীভাবে জানব? কীভাবে প্রমাণ করব যে এটি কাজ করে?

স্তর	প্রধান দক্ষতা	প্রমাণ	বইয়ের অধ্যায়
ভিত্তি	Variable, Function, Object, Async	ছোট Utility ও Unit Test	১-১৪
Application	DOM, API, Validation, State	ব্যবহারযোগ্য UI ও Error অবস্থা	১৫-১৮
Backend	HTTP, Auth, Database	লিখিত API ও Integration Test	২০, ২৫, ২৬
Engineering	Type, Test, নিরাপত্তা, CI	পুনরুৎপাদনযোগ্য Build, Code Review	২১-২৪, ২৮
Architecture	Scale, পর্যবেক্ষণ, ব্যর্থতার ধরন	ADR, চিত্র, Incident-পরিকল্পনা	২৭-২৯
পেশাদার	সহযোগিতা ও যোগাযোগ	Portfolio, README, সিদ্ধান্তের ব্যাখ্যা	৩০

৩০.২ Capstone: TravelMaster Pro-র Requirement

একটি ভ্রমণ সেবায় ব্যবহারকারী Package দেখবেন, Account খুলবেন, Login করবেন, Booking করবেন, নিজের Booking দেখবেন ও বাতিল করবেন।

Functional Requirement

১. Public Catalog-এ Package-এর নাম, দাম (টাকা) ও খালি আসন।
২. নিবন্ধনে নাম, Email ও Password যাচাই।
৩. Login-এর পরে একটি স্বল্পমেয়াদি Bearer Session Token।
৪. Booking-এর দাম **Server-এর Catalog** থেকে হিসাব; Client-এর পাঠানো দাম বিশ্বাস করা যাবে না।
৫. একই Idempotency Key-তে একই অনুরোধ আবার এলে দ্বিতীয়বার আসন কমবে না।
৬. নিজের Booking দেখা ও বাতিল করা যাবে; অন্যের Booking-এ প্রবেশ নয়।
৭. বাতিলে আসন ঠিক একবার ফেরত।
৮. Test-এ সফল পথ, ব্যর্থ পথ ও প্রান্তিক ক্ষেত্র।

Non-functional Requirement

- Error-এ Password বা ভেতরের Stack Trace কখনো নয়।
- JSON Body-র আকার সীমিত; অবৈধ JSON-এ 400।
- Token-এর মেয়াদ শেষে 401।
- সব HTTP Response একই চুক্তিতে: `{ data }` অথবা `{ error: { code, message } }`।
- Domain ও Transport স্তর আলাদা।
- একসঙ্গে আসা Booking-এ Overbooking নয়।
- README-তে Setup, সীমাবদ্ধতা ও Production-এর ঘাটতি স্পষ্ট।

> **নোট:** v30-এর Capstone-এ সব অবস্থা একটি Process-এর Memory-তে ছিল এবং Browser UI তৈরি বা পরীক্ষা করা হয়নি। এই Premium সংস্করণে TravelMaster Pro আসল SQLite Database (`node:sqlite`) ব্যবহার করে, Inventory Transaction-এ সুরক্ষিত, এবং একটি Browser Frontend আছে যা QA-তে আসল Headless Chrome-এ চালানো হয়েছে। **Payment অংশ বাস্তবায়িত নয়**—Booking সরাসরি নিশ্চিত হয়; এটি অনুচ্ছেদ ৩০.১০-এ স্পষ্টভাবে চিহ্নিত।

৩০.৩ Architecture: যা বানানো হয়েছে, আর যা লক্ষ্য

Folder-এর কাঠামো

```
Browser (public/app.mjs)
        |
        | HTTP + JSON + Bearer Token   (Production: HTTPS)
        v
  Transport: src/http.mjs   Route, JSON, Status, Security Header
        |
        v
  Domain: src/service.mjs   Validation, Auth, দাম, Idempotency, মালিকানা
        |
        v
  SQLite: src/db.mjs        users, sessions, packages, bookings, audit_events
                            CHECK, UNIQUE, FOREIGN KEY, Transaction

বানানো: একটি Process, একটি SQLite File (WAL Mode)
লক্ষ্য:  একাধিক Instance, PostgreSQL, Payment Gateway, Redis-এ Rate Limit
```

এই বিভাজনের কারণ: Domain (`service.mjs`) HTTP-র কিছুই জানে না, তাই তার প্রতিটি নিয়ম Network ছাড়াই Test করা যায়; Transport (`http.mjs`) কোনো ব্যবসার নিয়ম জানে না, শুধু অনুবাদ করে; আর Database-এর Constraint শেষ পাহারাদার—Code-এ Bug থাকলেও Schema অবৈধ অবস্থা আটকায়।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-master-pro/
├── package.json        start, demo, test, check, typecheck
├── README.md           চালানো, API, Requirement থেকে প্রমাণ, সীমাবদ্ধতা
├── tsconfig.json       checkJs + strict (JSDoc Type)
├── main.mjs            Environment পড়া, Server চালু, Signal
├── demo.mjs            HTTP দিয়ে প্রতিটি নিয়ম
├── public/             index.html, app.mjs, styles.css
├── src/
│   ├── errors.mjs      Error Code ও HTTP Status
│   ├── validation.mjs  Trust Boundary
│   ├── passwords.mjs   scrypt, Salt, সমান-সময়ের তুলনা
│   ├── db.mjs          Schema, Seed, Transaction
│   ├── service.mjs     Domain
│   ├── http.mjs        Transport
│   └── server.mjs      সব জোড়া লাগানো
└── tests/              ১৭টি Test
```

API চুক্তি

Method	Path	Auth	প্রত্যাশিত ফল
GET	`/health`	না	200
GET	`/api/packages`	না	200 Catalog
POST	`/api/register`	না	201; ভুল Input 400; একই Email 409
POST	`/api/login`	না	200 Token; ভুল 401; বারবার ভুলে 429
POST	`/api/logout`	হ্যাঁ	200, Session বাতিল
POST	`/api/bookings`	হ্যাঁ	201 নতুন; একই Key 200; ভিন্ন অনুরোধ বা আসন নেই 409
GET	`/api/bookings`	হ্যাঁ	200 নিজের Booking
PATCH	`/api/bookings/:id/cancel`	হ্যাঁ	200; অন্যের বা অজানা 404

v30-এর নকশায় Logout Token যাচাই না করেই Session বাতিল করত; এখানে Logout-এর আগে Token যাচাই হয়, অবৈধ Token-এ 401।

৩০.৪ মূল সিদ্ধান্ত ১: Trust Boundary-তে যাচাই

`typeof travelers === 'number'` যথেষ্ট নয়:

trust-boundary.mjs · JavaScript — চলবে: Node.js

```
import { validateBooking } from '../project/travel-master-pro/src/validation.mjs';

// typeof দিয়ে যাচাই যথেষ্ট নয়: এগুলো সবই "number"।
const suspicious = [2, Number.NaN, Infinity, 1.5, -3, 0, 11, '2'];
for (const travelers of suspicious) {
  const naive = typeof travelers === 'number';
  let strict;
  try {
    validateBooking({ packageId: 'BT-4D', travelers,
      idempotencyKey: 'demo-key-0001' });
    strict = 'গ্রহণ';
  } catch (error) {
    strict = `বাতিল (${error.details.join(', ')})`;
  }
  const shown = typeof travelers === 'string' ? `'${travelers}'`
    : String(travelers);
  console.log(shown.padEnd(8), `typeof: ${naive}`.padEnd(14), strict);
}
```

Output

```
2         typeof: true   গ্রহণ
NaN       typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
Infinity  typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
1.5       typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
-3        typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
0         typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
11        typeof: true   বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
'2'       typeof: false  বাতিল (travelers: ১-১০ পূর্ণসংখ্যা)
```

`NaN`, `Infinity`, `1.5` ও `-3` —সবগুলোর `typeof` "number"। ১.৫ জন যাত্রীর দাম বা ঋণাত্মক যাত্রীর জন্য আসন **বাড়ানো**—এগুলো শুধু তাত্ত্বিক নয়, আসল আক্রমণ। Validation Module:

validation.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// @ts-check
import { AppError } from './errors.mjs';

/** @param {unknown} value */
const text = (value) => (typeof value === 'string' ? value.trim() : '');

/**
 * Trust Boundary: বাইরে থেকে আসা প্রতিটি মান এখানে যাচাই ও স্বাভাবিক হয়।
 * @param {any} input
 * @returns {{ name: string, email: string, password: string }}
 */
export function validateRegistration(input) {
  const name = text(input?.name);
  const email = text(input?.email).toLowerCase();
  const password = typeof input?.password === 'string' ? input.password : '';
  const details = [];
  if (name.length < 2 || name.length > 60) details.push('name: ২-৬০ অক্ষর');
  // শুধু মৌলিক Format; Email-এর মালিকানা যাচাই আলাদা কাজ (Verification Link)।
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || email.length > 254) {
    details.push('email: সঠিক Format');
  }
  if (password.length < 12 || password.length > 128) {
    details.push('password: ১২-১২৮ অক্ষর');
  }
  if (details.length) throw new AppError('VALIDATION', 'Input ঠিক নয়', details);
  return { name, email, password };
}

// ...
```

Email-এর Regular Expression শুধু মৌলিক Format দেখে; Email-টি সত্যিই ব্যবহারকারীর কি না, তা জানার একমাত্র উপায় একটি Verification Link। Email ছোট হাতের অক্ষরে স্বাভাবিক করা হয়, তাই `RAFI@EXAMPLE.COM` দিয়ে দ্বিতীয় Account খোলা যায় না। Password-এ জটিলতার নিয়মের চেয়ে দৈর্ঘ্য বেশি কার্যকর—১২ অক্ষরের সীমা। Booking-এর যাচাই:

validation.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

/**
 * @param {any} input
 * @returns {{ packageId: string, travelers: number, idempotencyKey: string }}
 */
export function validateBooking(input) {
  const packageId = text(input?.packageId);
  const travelers = input?.travelers;
  const idempotencyKey = text(input?.idempotencyKey);
  const details = [];
  if (!/^[A-Z0-9-]{2,20}$/.test(packageId)) details.push('packageId');
  // NaN, Infinity, ভগ্নাংশ, ঋণাত্মক, String: সবই বাদ।
  if (!Number.isInteger(travelers) || travelers < 1 || travelers > 10) {
    details.push('travelers: ১-১০ পূর্ণসংখ্যা');
  }
  if (!/^[\w-]{8,64}$/.test(idempotencyKey)) details.push('idempotencyKey');
  if (details.length) throw new AppError('VALIDATION', 'Input ঠিক নয়', details);
  return { packageId, travelers, idempotencyKey };
}
```

প্রতিটি `// @ts-check` ও JSDoc Type (`@returns`) TypeScript Compiler-কে এই File-গুলো strict Mode-এ যাচাই করতে দেয়, কোনো Build ধাপ ছাড়াই (অধ্যায় ২৪)। Validation-এর পরে Domain জানে `travelers` একটি `number`—আর Compiler তা নিশ্চিত করে।

৩০.৫ মূল সিদ্ধান্ত ২: Password ও Session

passwords.mjs · JavaScript চলবে: Node.js

```
// @ts-check
import { randomBytes, scryptSync, timingSafeEqual } from 'node:crypto';

// scrypt: ইচ্ছাকৃতভাবে ধীর ও Memory-ভারী, তাই চুরি যাওয়া Hash থেকে
// অনুমান করে Password বের করা ব্যয়বহুল। প্রতিটি ব্যবহারকারীর আলাদা Salt।
const PARAMS = { N: 16384, r: 8, p: 1 };
const KEY_LENGTH = 32;

/** @param {string} password */
export function hashPassword(password) {
```

```
  const salt = randomBytes(16);
  const hash = scryptSync(password, salt, KEY_LENGTH, PARAMS);
  return `scrypt$${salt.toString('base64')}$${hash.toString('base64')}`;
}

/**
 * @param {string} password
 * @param {string} stored
 */
export function verifyPassword(password, stored) {
  const [scheme, salt, hash] = stored.split('$');
  if (scheme !== 'scrypt' || !salt || !hash) return false;
  const expected = Buffer.from(hash, 'base64');
  const actual = scryptSync(password, Buffer.from(salt, 'base64'),
    expected.length, PARAMS);
  // সমান সময়ে তুলনা: কতগুলো Byte মিলল, তা সময় দেখে বোঝা যায় না।
  return timingSafeEqual(actual, expected);
}

// অজানা Email-এও একই পরিমাণ কাজ, যাতে উত্তরের সময় দেখে বোঝা না যায়
// কোন Email নিবন্ধিত।
export const DUMMY_HASH = hashPassword('not-a-real-password-for-timing');
```

password-hash.mjs · JavaScript — চলবে: Node.js

```
import { hashPassword, verifyPassword }
  from '../project/travel-master-pro/src/passwords.mjs';

const password = 'long-password-123';
const first = hashPassword(password);
const second = hashPassword(password);

// একই Password, ভিন্ন Salt, তাই ভিন্ন Hash: দুই ব্যবহারকারীর একই
// Password থাকলেও Database দেখে তা বোঝা যায় না।
console.log('গঠন:', first.split('$').map((part, i) => (i === 0 ? part
  : `${part.length} অক্ষর`)).join(' | '));
console.log('দুটি Hash এক?', first === second);
console.log('সঠিক Password:', verifyPassword(password, first));
console.log('ভুল Password:', verifyPassword('long-password-124', first));
console.log('Hash-এ Password আছে?', first.includes(password));
```

Output

```
গঠন: scrypt | 24 অক্ষর | 44 অক্ষর
দুটি Hash এক? false
সঠিক Password: true
ভুল Password: false
Hash-এ Password আছে? false
```

Stored Hash-এ তিনটি অংশ: পদ্ধতির নাম, Salt ও Hash। পদ্ধতির নাম রাখা হয় যাতে ভবিষ্যতে Argon2id-এ গেলে পুরোনো Hash চেনা যায় এবং পরের Login-এ নতুন পদ্ধতিতে হালনাগাদ করা যায়।

Session: Login-এর পর ৩২ Byte-এর এলোমেলো Token Client পায়, কিন্তু Database-এ থাকে শুধু তার SHA-256 Hash ও মেয়াদ। Database চুরি গেলেও Hash থেকে Token ফেরানো যায় না, তাই চোর কারও হয়ে Login করতে পারে না। প্রতিটি অনুরোধে Token-এর Hash খুঁজে মেয়াদ যাচাই; Logout সেই সারি মুছে দেয়। ভুল Password ও অজানা Email—দুটিতেই একই বার্তা ("Email বা Password ভুল") এবং একই পরিমাণ কাজ (`DUMMY_HASH`), যাতে কেউ বার্তা বা সময় দেখে বুঝতে না পারে কোন Email নিবন্ধিত। পাঁচবার ভুলের পরে ১৫ মিনিট 429—Password অনুমান (Credential Stuffing) ধীর করতে।

৩০.৬ মূল সিদ্ধান্ত ৩: Database, দাম, Idempotency ও Inventory

Schema-ই শেষ পাহারাদার:

db.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
// @ts-check
import { DatabaseSync } from 'node:sqlite';

// Schema-ই শেষ পাহারাদার: Code-এ Bug থাকলেও seats কখনো ঋণাত্মক নয়,
// আর একই ব্যবহারকারীর একই idempotency_key দুটি Booking হতে পারে না।
const SCHEMA = `
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS sessions (
  token_hash TEXT PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  expires_at INTEGER NOT NULL);
CREATE TABLE IF NOT EXISTS packages (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  price_bdt INTEGER NOT NULL CHECK (price_bdt > 0),
  seats INTEGER NOT NULL CHECK (seats >= 0));
CREATE TABLE IF NOT EXISTS bookings (
  id INTEGER PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  package_id TEXT NOT NULL REFERENCES packages(id),
  travelers INTEGER NOT NULL CHECK (travelers BETWEEN 1 AND 10),
  total_bdt INTEGER NOT NULL CHECK (total_bdt > 0),
  status TEXT NOT NULL CHECK (status IN ('CONFIRMED', 'CANCELLED')),
  idempotency_key TEXT NOT NULL,
  created_at TEXT NOT NULL,
```

```
  UNIQUE (user_id, idempotency_key));
CREATE INDEX IF NOT EXISTS bookings_by_user ON bookings (user_id, id);
CREATE TABLE IF NOT EXISTS audit_events (
  id INTEGER PRIMARY KEY,
  type TEXT NOT NULL,
  user_id INTEGER,
  detail TEXT NOT NULL,
  at TEXT NOT NULL);`;

// ...
```

`CHECK (seats >= 0)` মানে Code-এ Bug থাকলেও আসন ঋণাত্মক হতে পারে না—Database নিজেই লেখা প্রত্যাখ্যান করে। `UNIQUE (user_id, idempotency_key)` মানে একই ব্যবহারকারীর একই Key দুটি Booking হতে পারে না, এমনকি দুটি Server একসঙ্গে চেষ্টা করলেও।

db.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

export const SEED_PACKAGES = [
  ['BT-4D', 'Bandarban Trek, 8 দিন', 12_500, 12],
  ['CXB-3D', "Cox's Bazar Beach, ৩ দিন", 9_800, 20],
  ['SYL-2D', 'Sylhet চা-বাগান, ২ দিন', 6_500, 2],
  ['SDB-3D', 'Sundarbans Boat Safari, ৩ দিন', 15_500, 8],
];

/** @param {string} [path] */
export function openDatabase(path = ':memory:') {
  const db = new DatabaseSync(path);
  db.exec('PRAGMA foreign_keys = ON; PRAGMA busy_timeout = 5000;');
  if (path !== ':memory:') db.exec('PRAGMA journal_mode = WAL;');
  db.exec(SCHEMA);
  const insert = db.prepare(`INSERT OR IGNORE INTO packages
    (id, name, price_bdt, seats) VALUES (?, ?, ?, ?)`);
  for (const row of SEED_PACKAGES) insert.run(...row);
  return db;
}

// ...
```

`foreign_keys` প্রতিটি সংযোগে চালু করতে হয় (অধ্যায় ২৬); `busy_timeout` অন্য সংযোগ লিখছে এমন সময়ে সঙ্গে সঙ্গে ব্যর্থ না হয়ে অপেক্ষা করতে দেয়; WAL Mode পড়া ও লেখা একসঙ্গে চলতে দেয়।

db.mjs · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

/**
```

```
 * BEGIN IMMEDIATE: লেখার Lock শুরুতেই, তাই দুটি সংযোগ একই সঙ্গে
 * "আসন আছে" দেখে দুজনেই বিক্রি করতে পারে না।
 * @template T
 * @param {DatabaseSync} db
 * @param {() => T} work
 * @returns {T}
 */
export function transaction(db, work) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = work();
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}
```

কেন "পড়ে, তারপর লেখা" বিপজ্জনক: দুটি অনুরোধ একসঙ্গে শেষ দুটি আসনের দুটি করে চাইলে কী হয়:

race-condition.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// দুটি অনুরোধ একসঙ্গে শেষ ২টি আসনের ২টি করে চায়।
// ভুল পথ: পড়া, তারপর (await-এর ফাঁকে) লেখা।
const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE packages (id TEXT PRIMARY KEY, seats INTEGER);
  INSERT INTO packages VALUES ('SYL-2D', 2);`);
const reset = () => db.exec("UPDATE packages SET seats = 2");
const seats = () => db.prepare('SELECT seats FROM packages').get().seats;
const pause = () => new Promise((resolve) => setTimeout(resolve, 10));

async function checkThenAct(want) {
  const available = seats();
  await pause(); // যেমন Payment Gateway-কে জিজ্ঞাসা, বা অন্য কোনো await
  if (available < want) return 'SOLD_OUT';
  db.prepare('UPDATE packages SET seats = ? WHERE id = ?')
    .run(available - want, 'SYL-2D');
  return 'BOOKED';
}

async function conditionalUpdate(want) {
  await pause();
  // যাচাই ও কমানো একটি Statement-এ: মাঝখানে কেউ ঢুকতে পারে না।
  const { changes } = db.prepare(`UPDATE packages SET seats = seats - ?
    WHERE id = ? AND seats >= ?`).run(want, 'SYL-2D', want);
  return changes ? 'BOOKED' : 'SOLD_OUT';
}
```

‘‘

```
for (const strategy of [checkThenAct, conditionalUpdate]) {
  reset();
  const results = await Promise.all([strategy(2), strategy(2)]);
  console.log(`${strategy.name}: ${results.join(', ')}; বাকি আসন ${seats()}`);
}
```

Output

```
checkThenAct: BOOKED, BOOKED; বাকি আসন 0
conditionalUpdate: BOOKED, SOLD_OUT; বাকি আসন 0
```

প্রথম পদ্ধতিতে দুজনেই "২টি আছে" পড়েছেন, তারপর দুজনেই বিক্রি করেছেন—চারটি আসন বিক্রি হলো দুটির জায়গায়, অথচ শেষে আসন শূন্য দেখায়, তাই ভুলটি চোখেও পড়ে না। দ্বিতীয় পদ্ধতিতে `WHERE seats >= ?` যাচাই ও কমানো একই Statement-এ, তাই দ্বিতীয়জন সঠিকভাবে SOLD_OUT পান।

src/service.mjs

প্রথম অংশ: Import, Token-এর Hash ও Booking-এর রূপান্তর:

service.mjs · JavaScript অংশ ১ / ৬ (Part 1 of 6) চলবে: Node.js

```
// @ts-check
import { createHash, randomBytes } from 'node:crypto';
import { AppError } from './errors.mjs';
import { transaction } from './db.mjs';
import { hashPassword, verifyPassword, DUMMY_HASH } from './passwords.mjs';
import { validateRegistration, validateBooking } from './validation.mjs';

/** @typedef {import('node:sqlite').DatabaseSync} Database */
/** @typedef {{ id: number, name: string, email: string }} User */

/** @param {string} token */
const sha256 = (token) => createHash('sha256').update(token).digest('hex');

/** @param {any} row */
const toBooking = (row) => ({ id: row.id, packageId: row.package_id,
  travelers: row.travelers, totalBdt: row.total_bdt, status: row.status,
  createdAt: row.created_at });

/**
 * Domain Service: সব ব্যবসার নিয়ম এখানে; HTTP-র কিছু জানে না।
 * @param {{ db: Database, now?: () => number, tokenTtlMs?: number,
 *   maxFailedLogins?: number }} options
 */
export function createTravelService({ db, now = Date.now,
  tokenTtlMs = 30 * 60_000, maxFailedLogins = 5 }) {
  /** @type {Map<string, { count: number, since: number }>} */
```

```
  const failures = new Map();
  const iso = () => new Date(now()).toISOString();

// ...
```

Audit ও নিবন্ধন। Password-এর Hash তৈরি হয় Database-এ লেখার আগে; একই Email-এ UNIQUE Constraint ভঙ্গ হলে একটি বোধগম্য Error:

service.mjs · JavaScript অংশ ২ / ৬ (Part 2 of 6) চলবে: Node.js

```
// ...

  /**
   * @param {string} type
   * @param {number | null} userId
   * @param {object} detail
   */
  const audit = (type, userId, detail) => db.prepare(
    'INSERT INTO audit_events (type, user_id, detail, at) VALUES (?, ?, ?, ?)')
    .run(type, userId, JSON.stringify(detail), iso());

  /** @param {any} input @returns {User} */
  function register(input) {
    const { name, email, password } = validateRegistration(input);
    const passwordHash = hashPassword(password);
    try {
      const { lastInsertRowid } = db.prepare(`INSERT INTO users
        (name, email, password_hash, created_at) VALUES (?, ?, ?, ?)`)
        .run(name, email, passwordHash, iso());
      audit('user.registered', Number(lastInsertRowid), {});
      return { id: Number(lastInsertRowid), name, email };
    } catch (error) {
      if (/UNIQUE/.test(String(error))) {
        throw new AppError('EMAIL_TAKEN', 'এই Email আগেই নিবন্ধিত');
      }
      throw error;
    }
  }

// ...
```

Login: ভুলের সীমা, অজানা Email-এও Hash-যাচাই, এবং Database-এ শুধু Token-এর Hash:

service.mjs · JavaScript অংশ ৩ / ৬ (Part 3 of 6) চলবে: Node.js

```
// ...

  /** @param {any} input */
  function login(input) {
```

```
    const email = typeof input?.email === 'string'
      ? input.email.trim().toLowerCase() : '';
    const password = typeof input?.password === 'string' ? input.password : '';
    const recent = failures.get(email);
    if (recent && recent.count >= maxFailedLogins
      && now() - recent.since < 15 * 60_000) {
      throw new AppError('TOO_MANY_ATTEMPTS', 'কিছুক্ষণ পরে চেষ্টা করুন');
    }
    /** @type {any} */
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
    // অজানা Email-এও একই Hash-যাচাই: সময় দেখে কিছু বোঝা যায় না।
    const ok = verifyPassword(password, user?.password_hash ?? DUMMY_HASH);
    if (!user || !ok) {
      const entry = recent && now() - recent.since < 15 * 60_000
        ? recent : { count: 0, since: now() };
      failures.set(email, { ...entry, count: entry.count + 1 });
      audit('login.failed', user?.id ?? null, {});
      throw new AppError('UNAUTHORIZED', 'Email বা Password ভুল');
    }
    failures.delete(email);
    // Database-এ শুধু Token-এর Hash: Database ফাঁস হলেও Token ব্যবহারযোগ্য নয়।
    const token = randomBytes(32).toString('base64url');
    const expiresAt = now() + tokenTtlMs;
    db.prepare('INSERT INTO sessions VALUES (?, ?, ?)')
      .run(sha256(token), user.id, expiresAt);
    audit('login.succeeded', user.id, {});
    return { token, expiresAt: new Date(expiresAt).toISOString(),
      user: { id: user.id, name: user.name, email: user.email } };
  }

// ...
```

service.mjs · JavaScript অংশ 8 / ৬ (Part 4 of 6) চলবে: Node.js

```
// ...

  /** @param {string | undefined} token @returns {User} */
  function authenticate(token) {
    /** @type {any} */
    const row = token && db.prepare(`SELECT u.id, u.name, u.email,
      s.expires_at FROM sessions s JOIN users u ON u.id = s.user_id
      WHERE s.token_hash = ?`).get(sha256(token));
    if (!row || row.expires_at <= now()) {
      throw new AppError('UNAUTHORIZED', 'Login প্রয়োজন');
    }
    return { id: row.id, name: row.name, email: row.email };
  }

  /** @param {string | undefined} token */
```

```
  function logout(token) {
    const user = authenticate(token);
    db.prepare('DELETE FROM sessions WHERE token_hash = ?')
      .run(sha256(/** @type {string} */ (token)));
    audit('logout', user.id, {});
  }

  function listPackages() {
    return db.prepare('SELECT id, name, price_bdt AS priceBdt, seats '
      + 'FROM packages ORDER BY id').all().map((row) => ({ ...row }));
  }

// ...
```

`authenticate`-এ `expires_at <= now()` —ঠিক মেয়াদের মুহূর্তেই অবৈধ। এবার Capstone-এর হৃদয়, `createBooking`। পুরোটা একটি `BEGIN IMMEDIATE` Transaction-এ: আগে একই Key-র Booking খোঁজা; থাকলে অনুরোধ মিলিয়ে আগেরটি ফেরত (`replayed: true`) বা KEY_CONFLICT; না থাকলে শর্তযুক্ত UPDATE; আসন না কমলে Package আছে কি না দেখে SOLD_OUT বা NOT_FOUND; তারপর Catalog-এর দামে Booking:

service.mjs · JavaScript অংশ ৫ / ৬ (Part 5 of 6) চলবে: Node.js

```
// ...

  /** @param {number} userId @param {any} input */
  function createBooking(userId, input) {
    const { packageId, travelers, idempotencyKey } = validateBooking(input);
    return transaction(db, () => {
      /** @type {any} */
      const existing = db.prepare(`SELECT * FROM bookings
        WHERE user_id = ? AND idempotency_key = ?`).get(userId, idempotencyKey);
      if (existing) {
        if (existing.package_id !== packageId
          || existing.travelers !== travelers) {
          throw new AppError('KEY_CONFLICT',
            'একই idempotencyKey, কিন্তু ভিন্ন অনুরোধ');
        }
        return { booking: toBooking(existing), replayed: true };
      }
      // শর্তযুক্ত UPDATE: যাচাই ও কমানো একটি Statement-এ, তাই Race নেই।
      const { changes } = db.prepare(`UPDATE packages SET seats = seats - ?
        WHERE id = ? AND seats >= ?`).run(travelers, packageId, travelers);
      if (changes === 0) {
        const found = db.prepare('SELECT 1 FROM packages WHERE id = ?')
          .get(packageId);
        throw found ? new AppError('SOLD_OUT', 'যথেষ্ট আসন নেই')
          : new AppError('NOT_FOUND', 'Package পাওয়া যায়নি');
      }
      // দাম সব সময় Server-এর Catalog থেকে; Client-এর পাঠানো দাম উপেক্ষিত।
```

```
      /** @type {any} */
      const pkg = db.prepare('SELECT price_bdt FROM packages WHERE id = ?')
        .get(packageId);
      const { lastInsertRowid } = db.prepare(`INSERT INTO bookings
        (user_id, package_id, travelers, total_bdt, status, idempotency_key,
        created_at) VALUES (?, ?, ?, ?, 'CONFIRMED', ?, ?)`)
        .run(userId, packageId, travelers, pkg.price_bdt * travelers,
          idempotencyKey, iso());
      audit('booking.created', userId, { bookingId: Number(lastInsertRowid) });
      const row = db.prepare('SELECT * FROM bookings WHERE id = ?')
        .get(lastInsertRowid);
      return { booking: toBooking(row), replayed: false };
    });
  }

// ...
```

তালিকা ও বাতিল। বাতিলের Query-তে `AND user_id = ?` —মালিকানা যাচাই আলাদা ধাপ নয়, Query-রই অংশ, তাই ভুলে বাদ পড়ার সুযোগ নেই। আগে বাতিল হয়ে থাকলে কিছুই না বদলে একই ফল:

service.mjs · JavaScript অংশ ৬ / ৬ (Part 6 of 6) চলবে: Node.js

```
// ...

  /** @param {number} userId */
  function listBookings(userId) {
    return db.prepare('SELECT * FROM bookings WHERE user_id = ? ORDER BY id')
      .all(userId).map(toBooking);
  }

  /** @param {number} userId @param {number} bookingId */
  function cancelBooking(userId, bookingId) {
    return transaction(db, () => {
      /** @type {any} */
      const row = db.prepare(`SELECT * FROM bookings
        WHERE id = ? AND user_id = ?`).get(bookingId, userId);
      // অন্যের Booking-ও "পাওয়া যায়নি": অস্তিত্বের তথ্যও ফাঁস নয়।
      if (!row) throw new AppError('NOT_FOUND', 'Booking পাওয়া যায়নি');
      if (row.status === 'CANCELLED') return toBooking(row); // একবারই ফেরত
      db.prepare("UPDATE bookings SET status = 'CANCELLED' WHERE id = ?")
        .run(bookingId);
      db.prepare('UPDATE packages SET seats = seats + ? WHERE id = ?')
        .run(row.travelers, row.package_id);
      audit('booking.cancelled', userId, { bookingId });
      return toBooking({ ...row, status: 'CANCELLED' });
    });
  }

  return { register, login, authenticate, logout, listPackages,
```

```
    createBooking, listBookings, cancelBooking };
}
```

৩০.৭ Transport: src/http.mjs

http.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
// @ts-check
import http from 'node:http';
import { readFile } from 'node:fs/promises';
import { AppError } from './errors.mjs';

/**
 * @typedef {ReturnType<typeof import('./service.mjs').createTravelService>}
 *   Service
 */
/** @typedef {[number, unknown]} Result */

const STATIC = {
  '/': ['index.html', 'text/html'],
  '/app.mjs': ['app.mjs', 'text/javascript'],
  '/styles.css': ['styles.css', 'text/css'],
};

// Browser-কে কঠোর নিয়ম: শুধু নিজের Script ও Style, অন্য Site-এ Frame নয়।
const SECURITY_HEADERS = {
  'X-Content-Type-Options': 'nosniff',
  'Referrer-Policy': 'no-referrer',
  'Content-Security-Policy': "default-src 'self'; script-src 'self'; "
    + "style-src 'self'; connect-src 'self'; img-src 'self' data:; "
    + "frame-ancestors 'none'; base-uri 'none'; form-action 'self'",
};

// ...
```

Content-Security-Policy Browser-কে বলে শুধু নিজের Domain-এর Script ও Style চালাতে—কেউ কোনোভাবে একটি `<script>` ঢুকিয়ে দিলেও (XSS) Browser তা চালাবে না। `frame-ancestors 'none'` অন্য Site-কে এই Page Frame-এ রাখতে দেয় না (Clickjacking)।

http.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

/** @param {http.IncomingMessage} req @param {number} limit */
async function readJson(req, limit) {
  if (!/^application\/json\b/.test(req.headers['content-type'] ?? '')) {
    throw new AppError('UNSUPPORTED_MEDIA_TYPE', 'JSON পাঠান');
```

```
  }
  let size = 0;
  const chunks = [];
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new AppError('TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new AppError('BAD_JSON', 'অবৈধ JSON');
  }
}

/** @param {http.IncomingMessage} req */
const bearer = (req) => /^Bearer ([\w-]+)$/
  .exec(req.headers.authorization ?? '')?.[1];

// ...
```

Route-এর তালিকা। Booking তৈরিতে আগে Authentication, তারপর Body পড়া—পরিচয়হীন কারও Input পড়ার কাজই হয় না। একই Key-তে দ্বিতীয় অনুরোধে Status 200, প্রথমবারে 201—Client চাইলে পার্থক্য বুঝতে পারে:

http.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

/**
 * Transport স্তর: Route, JSON, Status ও Header। ব্যবসার নিয়ম Service-এ।
 * @param {{ service: Service, bodyLimit?: number,
 *   log?: (event: string, fields: object) => void }} options
 */
export function createHttpServer({ service, bodyLimit = 4096,
  log = () => {} }) {
  const user = (/** @type {http.IncomingMessage} */ req) =>
    service.authenticate(bearer(req));
  const body = (/** @type {http.IncomingMessage} */ req) =>
    readJson(req, bodyLimit);

  /** @type {[string, RegExp, (req: http.IncomingMessage,
   *   params: string[]) => Promise<Result>][]} */
  const routes = [
    ['GET', /^\/health$/, async () => [200, { status: 'ok' }]],
    ['GET', /^\/api\/packages$/, async () => [200, service.listPackages()]],
    ['POST', /^\/api\/register$/,
      async (req) => [201, service.register(await body(req))]],
    ['POST', /^\/api\/login$/,
      async (req) => [200, service.login(await body(req))]],
```

```
    ['POST', /^\/api\/logout$/, async (req) => {
      service.logout(bearer(req));
      return [200, { loggedOut: true }];
    }],
    ['GET', /^\/api\/bookings$/,
      async (req) => [200, service.listBookings(user(req).id)]],
    // আগে Authentication, তারপর Body: পরিচয় ছাড়া কোনো Input পড়া নয়।
    ['POST', /^\/api\/bookings$/, async (req) => {
      const { id } = user(req);
      const { booking, replayed } = service.createBooking(id, await body(req));
      return [replayed ? 200 : 201, booking];
    }],
    ['PATCH', /^\/api\/bookings\/(\d+)\/cancel$/, async (req, [bookingId]) =>
      [200, service.cancelBooking(user(req).id, Number(bookingId))]],
  ];

// ...
```

Static File, Route মেলানো এবং Error চুক্তি। `AppError` তার নিজের Status ও Code পায়; বাকি সব 500, সাধারণ বার্তা, আর বিবরণ শুধু Log-এ। প্রতিটি API Response-এ `Cache-Control: no-store`, যাতে কোনো Proxy বা Browser Cache ব্যক্তিগত তথ্য জমা না রাখে:

http.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  return http.createServer(async (req, res) => {
    const { pathname } = new URL(req.url ?? '/', 'http://localhost');
    try {
      const file = req.method === 'GET'
        && STATIC[/** @type {keyof typeof STATIC} */ (pathname)];
      if (file) {
        const content = await readFile(
          new URL(`../public/${file[0]}`, import.meta.url));
        res.writeHead(200, { ...SECURITY_HEADERS,
          'Content-Type': `${file[1]}; charset=utf-8` });
        return void res.end(content);
      }
      for (const [method, pattern, handler] of routes) {
        const match = pattern.exec(pathname);
        if (match && req.method === method) {
          const [status, data] = await handler(req, match.slice(1));
          return send(res, status, { data });
        }
      }
      throw new AppError('NOT_FOUND', 'Route পাওয়া যায়নি');
    } catch (error) {
      if (error instanceof AppError) {
        const { code, message, details } = error;
```

```
        return send(res, error.status, { error: { code, message,
          ...(details.length ? { details } : {}) } });
      }
      // অপ্রত্যাশিত: পুরো বিবরণ Log-এ, Client পায় শুধু সাধারণ বার্তা।
      log('http.unhandled', { path: pathname, error: String(error),
        stack: error instanceof Error ? error.stack : undefined });
      return send(res, 500, { error: { code: 'INTERNAL',
        message: 'Server-এ সমস্যা' } });
    }
  });
}

/**
 * @param {http.ServerResponse} res
 * @param {number} status
 * @param {object} body
 */
function send(res, status, body) {
  res.writeHead(status, { ...SECURITY_HEADERS,
    'Content-Type': 'application/json; charset=utf-8',
    'Cache-Control': 'no-store' });
  res.end(JSON.stringify(body));
}
```

`server.mjs` তিনটি স্তর জোড়া লাগায় (Composition Root), আর `main.mjs` Environment থেকে Port ও Database-এর Path পড়ে:

server.mjs · JavaScript — চলবে: Node.js

```
// @ts-check
import { openDatabase } from './db.mjs';
import { createTravelService } from './service.mjs';
import { createHttpServer } from './http.mjs';

/**
 * Database, Service ও HTTP একসঙ্গে জোড়া লাগানো (Composition Root)।
 * @param {{ port?: number, host?: string, dbPath?: string,
 *   now?: () => number, tokenTtlMs?: number,
 *   log?: (event: string, fields: object) => void }} [options]
 */
export async function startServer({ port = 0, host = '127.0.0.1',
  dbPath = ':memory:', now, tokenTtlMs, log } = {}) {
  const db = openDatabase(dbPath);
  const service = createTravelService({ db, now, tokenTtlMs });
  const server = createHttpServer({ service, log });
  await new Promise((resolve) => server.listen(port, host, () => resolve(0)));
  const address = /** @type {import('node:net').AddressInfo} */ (
    server.address());
  return {
```

```
    url: `http://${host}:${address.port}`,
    server, service, db,
    close: () => new Promise((resolve) => {
      server.closeAllConnections();
      server.close(() => {
        db.close();
        resolve(0);
      });
    }),
  };
}
```

৩০.৮ Frontend: public/app.mjs

Frontend একটি সাধারণ ES Module, কোনো Framework ছাড়া। প্রথম অংশ: অবস্থা, Error-এর বন্ধুত্বপূর্ণ বার্তা এবং সব API ডাকের একটি Function:

app.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Browser

```
// Token শুধু Memory-তে (Page Reload-এ Logout)। localStorage-এ রাখলে কোনো XSS
// Script তা পড়তে পারত; Production-এ HttpOnly Cookie + CSRF সুরক্ষা ভালো।
const state = { token: null, user: null, pendingKey: null, busy: false };
const $ = (selector) => document.querySelector(selector);
const MESSAGES = {
  SOLD_OUT: 'দুঃখিত, এত আসন খালি নেই।',
  EMAIL_TAKEN: 'এই Email আগেই নিবন্ধিত।',
  KEY_CONFLICT: 'আগের অনুরোধের সঙ্গে মিলছে না; আবার চেষ্টা করুন।',
  TOO_MANY_ATTEMPTS: 'অনেকবার ভুল হয়েছে; কিছুক্ষণ পরে চেষ্টা করুন।',
};

function say(text, isError = false) {
  $('#message').textContent = text;
  $('#message').classList.toggle('error', isError);
}

// সব API ডাক এক জায়গায়: Token, JSON, Error-এর বার্তা।
async function api(path, { method = 'GET', body } = {}) {
  let res;
  try {
    res = await fetch(path, { method, headers: {
      ...(body && { 'Content-Type': 'application/json' }),
      ...(state.token && { Authorization: `Bearer ${state.token}` }) },
    body: body && JSON.stringify(body) });
  } catch {
    throw new Error('Network সমস্যা; সংযোগ দেখে আবার চেষ্টা করুন।');
  }
  const payload = await res.json();
```

```
  if (res.ok) return payload.data;
  const { code, message, details } = payload.error;
  // Login থাকা অবস্থায় 401 মানে Session শেষ; Login Form-এ তা ভুল Password।
  if (code === 'UNAUTHORIZED' && state.token) {
    signOut();
    throw new Error('Session শেষ হয়েছে; আবার Login করুন।');
  }
  const extra = details?.length ? ` (${details.join(', ')})` : '';
  throw new Error(MESSAGES[code] ?? `${message}${extra}`);
}

// ...
```

Token শুধু একটি JavaScript Variable-এ— `localStorage` -এ নয়। কারণ: কোনো XSS Script `localStorage` পড়তে পারে। মূল্য: Page Reload করলে আবার Login। Production-এ ভালো বিকল্প HttpOnly Cookie (JavaScript পড়তেই পারে না), সঙ্গে CSRF সুরক্ষা। আরেকটি বিষয়: Login অবস্থায় 401 মানে Session শেষ, কিন্তু Login Form-এ 401 মানে ভুল Password—দুটিকে আলাদা করা হয়েছে। প্রথম সংস্করণে দুটিই "Session শেষ হয়েছে" দেখাত; Browser End-to-End Test এই ভুল ধরেছে।

app.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Browser

```
// ...

const taka = (n) => `${n.toLocaleString('en-IN')} টাকা`;

async function loadPackages() {
  const packages = await api('/api/packages');
  $('#packages').replaceChildren(...packages.map((p) => {
    const item = document.createElement('li');
    item.textContent = `${p.name}: ${taka(p.priceBdt)}, আসন ${p.seats}`;
    return item;
  }));
  const select = $('#book select');
  const chosen = select.value;
  select.replaceChildren(...packages.map((p) => new Option(
    `${p.name} (${taka(p.priceBdt)})`, p.id, false, p.id === chosen)));
}

async function loadBookings() {
  const bookings = await api('/api/bookings');
  $('#bookings').replaceChildren(...bookings.map((b) => {
    const item = document.createElement('li');
    item.textContent = `#${b.id} ${b.packageId}, ${b.travelers} জন, `
      + `${taka(b.totalBdt)}, ${b.status} `;
    item.classList.toggle('cancelled', b.status === 'CANCELLED');
    if (b.status === 'CONFIRMED') {
      const cancel = document.createElement('button');
      cancel.type = 'button';
```

```
      cancel.textContent = 'বাতিল';
      cancel.dataset.id = b.id;
      item.append(cancel);
    }
    return item;
  }));
}

// ...
```

DOM-এ সব লেখা `textContent` দিয়ে, কখনো `innerHTML` নয়—Package-এর নাম বা Server-এর বার্তায় HTML থাকলেও তা Code হিসেবে চলবে না।

app.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Browser

```
// ...

function render() {
  $('#who').textContent = state.user
    ? `${state.user.name} হিসেবে Login` : 'অতিথি হিসেবে দেখছেন';
  $('#logout').hidden = !state.user;
  $('#account').hidden = Boolean(state.user);
  $('#booking').hidden = !state.user;
}

function signOut() {
  Object.assign(state, { token: null, user: null, pendingKey: null });
  render();
}

// প্রতিটি Form-এর জন্য একই কাঠামো: বার্তা মোছা, কাজ, Error দেখানো।
function onSubmit(selector, work) {
  $(selector).addEventListener('submit', async (event) => {
    event.preventDefault();
    say('');
    try {
      await work(new FormData(event.target), event.target);
    } catch (error) {
      say(error.message, true);
    }
  });
}

onSubmit('#register', async (form, element) => {
  await api('/api/register', { method: 'POST',
    body: Object.fromEntries(form) });
  element.reset();
  say('নিবন্ধন সম্পন্ন। এবার Login করুন।');
});
```

```
onSubmit('#login', async (form, element) => {
  const session = await api('/api/login', { method: 'POST',
    body: Object.fromEntries(form) });
  Object.assign(state, { token: session.token, user: session.user });
  element.reset();
  render();
  say(`স্বাগতম, ${session.user.name}।`); // আগে বার্তা: পরে এলে নতুন বার্তা মুছত
  await loadBookings();
});

// ...
```

Booking Form-এ দুই স্তরের Double-submit সুরক্ষা। Client-এ: `busy` থাকলে দ্বিতীয় Submit উপেক্ষিত, Button নিষ্ক্রিয়। Server-এ: `pendingKey` একবার তৈরি হয় এবং সফল না হওয়া পর্যন্ত একই থাকে—তাই Network-এ উত্তর হারিয়ে গেলে ব্যবহারকারী আবার চাপলে একই Key যায়, আর Server আগের Booking ফেরত দেয়। Form-এ কিছু বদলালে নতুন অনুরোধ, নতুন Key:

app.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Browser

```
// ...

// Double Submit: busy থাকলে দ্বিতীয়টি উপেক্ষিত। Network ব্যর্থ হয়ে আবার
// চেষ্টা করলে একই pendingKey যায়, তাই Server-ও দ্বিতীয় Booking করে না।
onSubmit('#book', async (form) => {
  if (state.busy) return;
  state.busy = true;
  $('#book-button').disabled = true;
  state.pendingKey ??= crypto.randomUUID();
  try {
    const booking = await api('/api/bookings', { method: 'POST', body: {
      packageId: form.get('packageId'),
      travelers: Number(form.get('travelers')),
      idempotencyKey: state.pendingKey } });
    state.pendingKey = null;
    say(`Booking #${booking.id} নিশ্চিত: ${taka(booking.totalBdt)}`);
    await Promise.all([loadBookings(), loadPackages()]);
  } finally {
    state.busy = false;
    $('#book-button').disabled = false;
  }
});

// Form-এর কিছু বদলালে নতুন অনুরোধ, তাই নতুন Key।
$('#book').addEventListener('input', () => { state.pendingKey = null; });

$('#bookings').addEventListener('click', async (event) => {
  const id = event.target.dataset?.id;
```

```
  if (!id) return;
  try {
    await api(`/api/bookings/${id}/cancel`, { method: 'PATCH' });
    say(`Booking #${id} বাতিল হয়েছে।`);
    await Promise.all([loadBookings(), loadPackages()]);
  } catch (error) {
    say(error.message, true);
  }
});

$('#logout').addEventListener('click', async () => {
  await api('/api/logout', { method: 'POST' }).catch(() => {});
  signOut();
  say('Logout হয়েছে।');
});

render();
loadPackages().catch((error) => say(error.message, true));
```

HTML-এ প্রতিটি Input-এর `<label>`, বার্তার জায়গায় `role="status"` ও `aria-live="polite"` (Screen Reader বার্তা পড়ে শোনায়), এবং Keyboard দিয়ে সব কাজ করা যায় কারণ সব Control আসল `<button>`, `<input>` ও `<select>`। CSS-এ একটি Media Query ৭২০ Pixel-এর কম চওড়ায় দুই কলাম থেকে এক কলামে নামায়।

৩০.৯ Test-এর পিরামিড ও Quality Gate

স্তর	কী যাচাই করে	কোথায়	সংখ্যা
Unit	Validation-এর প্রতিটি প্রান্ত	`validation.test.mjs`	৩
Domain	দাম, Idempotency, Sold-out, মালিকানা, বাতিল	`service.test.mjs`	৫
নিরাপত্তা	Hash, একই বার্তা, Rate Limit, মেয়াদ, Logout	`auth.test.mjs`	৫
Integration (HTTP)	পুরো প্রবাহ, Status, Error চুক্তি, Header	`http.test.mjs`	৩
Concurrency	চারটি Thread, একই Database, Overbooking নয়	`concurrency.test.mjs`	১
Type	strict Mode-এ JSDoc Type	`npm run typecheck`	৭ File
Browser E2E	আসল Chrome-এ Form, Double-submit, Mobile	QA Script	১৪ যাচাই

Idempotency-র Domain Test:

service.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Idempotency: একই Key ও অনুরোধে একই Booking, আসন একবার কমে', () => {
  const { service, user, seats } = domain();
  const { user: rafi } = user('Rafi');
  const first = service.createBooking(rafi.id, order('key-retry-01'));
  const retry = service.createBooking(rafi.id, order('key-retry-01'));
  assert.equal(retry.booking.id, first.booking.id);
  assert.deepEqual([first.replayed, retry.replayed], [false, true]);
  assert.equal(seats('BT-4D'), 10);
  assert.throws(() => service.createBooking(rafi.id,
    order('key-retry-01', { travelers: 3 })), { code: 'KEY_CONFLICT' });
  // অন্য ব্যবহারকারীর একই Key আলাদা অনুরোধ
  const { user: mina } = user('Mina');
  assert.notEqual(service.createBooking(mina.id, order('key-retry-01'))
    .booking.id, first.booking.id);
});

// ...
```

আর সবচেয়ে কঠিন প্রমাণ—সত্যিকারের একসঙ্গে লেখা। চারটি `worker_threads`, প্রত্যেকে নিজস্ব Database সংযোগ খুলে একই File-এ দশবার করে Booking চেষ্টা করে, অথচ আসন মাত্র ২০টি:

concurrency.test.mjs · JavaScript চলবে: Node.js

```
// চারটি Thread, প্রত্যেকের নিজস্ব সংযোগ, একই SQLite File: ৪০টি চেষ্টা,
// আসন মাত্র ২০। ঠিক ২০টি Booking হতে হবে, একটিও বেশি নয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { Worker } from 'node:worker_threads';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase } from '../src/db.mjs';
import { PASSWORD } from './helpers.mjs';

test('একসঙ্গে Booking-এ Overbooking হয় না', async (t) => {
  const dir = mkdtempSync(join(tmpdir(), 'tmp-'));
  const dbPath = join(dir, 'race.db');
  openDatabase(dbPath).close(); // Schema ও Seed: CXB-3D-তে ২০ আসন
  t.after(() => rmSync(dir, { recursive: true, force: true }));

  const workers = [0, 1, 2, 3].map((index) => new Promise((resolve, reject) => {
    const worker = new Worker(new URL('./fixtures/booking-worker.mjs',
      import.meta.url), { workerData: { dbPath, index, attempts: 10,
      password: PASSWORD } });
    worker.once('message', resolve);
```

```
    worker.once('error', reject);
  }));
  const results = await Promise.all(workers);
  const booked = results.reduce((sum, r) => sum + r.booked, 0);
  const soldOut = results.reduce((sum, r) => sum + r.soldOut, 0);
  assert.deepEqual([booked, soldOut], [20, 20]);

  const db = openDatabase(dbPath);
  const { seats } = db.prepare(
    "SELECT seats FROM packages WHERE id = 'CXB-3D'").get();
  const { sold } = db.prepare(`SELECT SUM(travelers) AS sold FROM bookings
    WHERE package_id = 'CXB-3D' AND status = 'CONFIRMED'`).get();
  db.close();
  assert.deepEqual([seats, sold], [0, 20]);
});
```

ঠিক ২০টি Booking, ২০টি SOLD_OUT, শেষে আসন শূন্য এবং বিক্রীত যাত্রীর যোগফল ২০। এটি সম্ভব হয়েছে তিন স্তরের সুরক্ষায়: `BEGIN IMMEDIATE` (লেখার Lock শুরুতেই), শর্তযুক্ত UPDATE, এবং শেষ পাহারাদার `CHECK (seats >= 0)`।

Test পাস করা মানে পুরো নিরাপত্তা নিরীক্ষা বা Production-এর নির্ভুলতা প্রমাণিত নয়; এর অর্থ শুধু এই যে লিখিত Requirement-গুলো লিখিত পরিস্থিতিতে কাজ করে।

৩০.১০ Project: TravelMaster Pro চালানো ও প্রমাণ

Terminal

```
cd travel-master-pro
npm test
npm run demo
npm start
```

Demo HTTP দিয়ে প্রতিটি ব্যবসার নিয়ম দেখায়:

demo.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// HTTP দিয়ে পুরো Capstone-এর প্রতিটি নিয়ম, এক Process-এ (Memory Database)।
import { startServer } from './src/server.mjs';

const app = await startServer();
async function call(path, { method = 'GET', token, body } = {}) {
  const res = await fetch(`${app.url}${path}`, { method, headers: {
    ...(token && { Authorization: `Bearer ${token}` }),
    ...(body && { 'Content-Type': 'application/json' }) },
  body: body && JSON.stringify(body) });
  const json = await res.json();
```

```
  return { status: res.status, ...json };
}
const show = (label, { status, data, error }) => {
  const detail = error?.code ?? `${data.status}, ${data.totalBdt} টাকা`;
  console.log(`${label}: ${status} ${detail}`);
};

const people = {};
for (const name of ['Rafi', 'Mina']) {
  const email = `${name.toLowerCase()}@example.com`;
  await call('/api/register', { method: 'POST',
    body: { name, email, password: 'long-password-123' } });
  people[name] = (await call('/api/login', { method: 'POST',
    body: { email, password: 'long-password-123' } })).data.token;
}

// ...
```

demo.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const order = { packageId: 'BT-4D', travelers: 2, price: 1,
  idempotencyKey: 'demo-key-0001' };
const book = (token, body) => call('/api/bookings', { method: 'POST',
  token, body });
const first = await book(people.Rafi, order);
show('৬. Booking (Client price 1 পাঠিয়েছে)', first);
show('২. একই Key আবার', await book(people.Rafi, order));
show('৩. একই Key, ভিন্ন যাত্রী', await book(people.Rafi,
  { ...order, travelers: 3 }));
show('8. Token ছাড়া', await book(undefined, order));
show('৫. SYL-2D-তে ৩ জন', await book(people.Rafi,
  { packageId: 'SYL-2D', travelers: 3, idempotencyKey: 'demo-key-0002' }));
const cancelPath = `/api/bookings/${first.data.id}/cancel`;
show('৬. Mina বাতিল করতে চান', await call(cancelPath,
  { method: 'PATCH', token: people.Mina }));
show('৭. Rafi বাতিল করেন', await call(cancelPath,
  { method: 'PATCH', token: people.Rafi }));
const seats = (await call('/api/packages')).data
  .find((p) => p.id === 'BT-4D').seats;
console.log('BT-4D-এ আসন:', seats);
await app.close();
```

Output

```
১. Booking (Client price 1 পাঠিয়েছে): 201 CONFIRMED, 25000 টাকা
২. একই Key আবার: 200 CONFIRMED, 25000 টাকা
৩. একই Key, ভিন্ন যাত্রী: 409 KEY_CONFLICT
৪. Token ছাড়া: 401 UNAUTHORIZED
৫. SYL-2D-তে ৩ জন: 409 SOLD_OUT
৬. Mina বাতিল করতে চান: 404 NOT_FOUND
৭. Rafi বাতিল করেন: 200 CANCELLED, 25000 টাকা
BT-4D-এ আসন: 12
```

Client `price: 1` পাঠিয়েছিল, কিন্তু দাম ২ × ১২,৫০০ = ২৫,০০০ টাকা—Catalog থেকে। Mina-র জন্য অন্যের Booking "পাওয়া যায়নি" (404), "নিষিদ্ধ" (403) নয়—যাতে অস্তিত্বের তথ্যও ফাঁস না হয়। শেষে আসন আবার ১২: Booking-এ ২টি কমেছিল, বাতিলে ২টি ফিরেছে, দ্বিতীয় অনুরোধে কিছুই বদলায়নি।

Output

```
# tests 17
# pass 17
# fail 0
```

QA-তে যা চালানো হয়েছে: ১৭টি Test (Node.js 22.16.0); Demo; TypeScript 6.0.2 দিয়ে strict Type Check—`src/`-এর সাতটি File-এ শূন্য Error, এবং প্রমাণ হিসেবে একটি ইচ্ছাকৃত Type-ভুল সঠিকভাবে ধরা পড়েছে; এবং আসল Headless Chrome 155-এ Frontend-এর ১৪টি যাচাই: Catalog, নিবন্ধন, ভুল Password-এর বার্তা, Login, Server-এর দামে Booking, দুটি দ্রুত Submit-এ ঠিক একটি Booking, Catalog-এর আসন হালনাগাদ, Sold-out বার্তা, বাতিল ও আসন ফেরত, ৩৭৫ Pixel চওড়ায় কোনো আনুভূমিক Scroll নেই, Logout, এবং কোনো Console Error বা CSP লঙ্ঘন নেই।

Feature-এর সৎ হিসাব:

Feature	অবস্থা
Catalog, নিবন্ধন, Login, Logout	বাস্তবায়িত ও পরীক্ষিত
Server-নির্ধারিত দাম, Idempotency, Inventory Transaction	বাস্তবায়িত ও পরীক্ষিত (Concurrency-সহ)
নিজের Booking, বাতিল, মালিকানা	বাস্তবায়িত ও পরীক্ষিত
Token-এর মেয়াদ, Login-এর ভুলের সীমা	বাস্তবায়িত ও পরীক্ষিত (সীমা শুধু এক Process-এ)
Browser Frontend	বাস্তবায়িত; আসল Chrome-এ পরীক্ষিত
Payment, Refund, Webhook	**বাস্তবায়িত নয়**
Email যাচাই, Password Reset	**বাস্তবায়িত নয়**
HTTPS, Docker, CI, Monitoring	এই Project-এ নয় (অধ্যায় ২৮-এ আলাদাভাবে)

Feature	অবস্থা
Screen Reader-এ আসল পরীক্ষা	**করা হয়নি**; শুধু Semantic HTML ও ARIA

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`node:sqlite` পাওয়া যায় না	Node.js 22.5-এর পুরোনো	Node.js হালনাগাদ
"database is locked"	দীর্ঘ Transaction বা অন্য Process	Transaction ছোট; `busy_timeout`
Login-এর পর Reload করলে আবার Login	Token Memory-তে (নকশা অনুযায়ী)	স্বাভাবিক; অনুচ্ছেদ ৩০.৮
Browser Console-এ CSP Error	Inline Script বা Style যোগ করা হয়েছে	আলাদা File-এ সরান
বারবার 429	পাঁচবার ভুল Password	১৫ মিনিট পরে
Restart-এর পর Data নেই	`DB_PATH=:memory:`	File Path দিন

৩০.১১ Production-এর জন্য উন্নয়ন-পরিকল্পনা

১. **PostgreSQL:** একই Schema; `SELECT ... FOR UPDATE` বা শর্তযুক্ত UPDATE; Connection Pool (অধ্যায় ২৬)।

২. **Migration:** সংস্করণভিত্তিক File, Expand/Contract (অধ্যায় ২৬, ২৮)।

৩. **Payment:** Booking-এর অবস্থা PENDING_PAYMENT; Gateway-এর Idempotency-Key; Webhook-এর Signature যাচাই ও Replay সুরক্ষা; নির্দিষ্ট সময়ে Payment না হলে আসন ছাড়া; Refund ও প্রতিদিনের Reconciliation (অধ্যায় ২৯)।

৪. **Session:** HttpOnly, Secure, SameSite Cookie; CSRF Token; Token Rotation; Rate Limit একটি ভাগ-করা Store-এ।

৫. **পর্যবেক্ষণ:** Request ID, Structured Log, Latency ও Error-এর Metrics, SLO-ভিত্তিক Alert (অধ্যায় ২৮)।

৬. **Delivery:** Docker Image, CI, Staging, ধাপে ধাপে Rollout, Backup ও Restore Drill, Rollback (অধ্যায় ২৮)।

৭. **গোপনীয়তা:** ন্যূনতম তথ্য, সংরক্ষণের মেয়াদ, প্রবেশাধিকার নিয়ন্ত্রণ, Secret Manager।

৮. **Accessibility:** আসল Screen Reader ও শুধু-Keyboard ব্যবহারে পরীক্ষা।

কখনো এই Demo Server সরাসরি Internet-এ একটি আসল ভ্রমণ-Booking সেবা হিসেবে চালাবেন না।

৩০.১২ Architecture Decision Record

ADR একটি ছোট Document: কোন প্রেক্ষাপটে কী সিদ্ধান্ত, কোন বিকল্প বাদ গেল, এবং তার মূল্য কী। এক বছর পরে নতুন সহকর্মী জানতে চাইবেন "কেন SQLite?"—উত্তর লেখা থাকবে।

ADR-001: Database হিসেবে SQLite (`node:sqlite`)। প্রেক্ষাপট: পাঠককে কোনো Package বা Server Install ছাড়াই আসল Transaction ও Constraint দেখাতে হবে। সিদ্ধান্ত: Node.js-এর অন্তর্নির্মিত SQLite, WAL Mode, `BEGIN IMMEDIATE`। বিকল্প: in-memory Map (Transaction ও Constraint নেই, তাই বাদ); PostgreSQL (আলাদা Server লাগে)। মূল্য: একটি Server-এই সীমিত; একাধিক Instance-এ PostgreSQL লাগবে, তবে Schema ও Query প্রায় একই থাকবে।

ADR-002: Token Browser-এর Memory-তে। প্রেক্ষাপট: XSS-এর ঝুঁকি ও সরলতা। সিদ্ধান্ত: Bearer Token শুধু JavaScript Variable-এ, কঠোর CSP-সহ। বিকল্প: `localStorage` (XSS পড়তে পারে, বাদ); HttpOnly Cookie (সবচেয়ে নিরাপদ, কিন্তু CSRF সুরক্ষা লাগে—Production-এর পরিকল্পনায়)। মূল্য: Reload-এ আবার Login।

ADR-003: Framework ছাড়া Transport ও Frontend। প্রেক্ষাপট: শেখার Capstone, যেখানে প্রতিটি স্তর দৃশ্যমান হওয়া জরুরি। সিদ্ধান্ত: `node:http` ও সাধারণ DOM। মূল্য: বড় দলে বা বড় UI-তে Express/Fastify ও একটি UI Framework সময় বাঁচাবে; Domain Service যেহেতু HTTP থেকে আলাদা, সেই বদল সীমিত।

৩০.১৩ ১০০ নম্বরের মূল্যায়ন

ক্ষেত্র	নম্বর	যা দেখা হয়
Requirement ও User Flow	১০	প্রান্তিক ক্ষেত্র স্পষ্ট
JavaScript-এর নকশা	১৫	Module, Validation, স্তর-বিভাজন
API ও Authentication	১৫	সঠিক Status, মালিকানা
Data-র সামঞ্জস্য	১৫	Idempotency, Inventory, Transaction
Test ও Debugging	১৫	স্বয়ংক্রিয় ও অর্থপূর্ণ Test
নিরাপত্তা ও গোপনীয়তা	১০	হুমকি ও প্রতিরোধ
Deployment ও Monitoring-এর পরিকল্পনা	১০	প্রস্তুতির Checklist
Documentation ও যোগাযোগ	১০	README, ADR, Demo

ব্যাখ্যা: ৮৫-১০০ শক্তিশালী Capstone; ৭০-৮৪ ভালো ভিত্তি; ৫০-৬৯ নির্দিষ্ট উন্নতি দরকার; ৫০-এর নিচে আগে Requirement ও Test। এটি শেখার মানদণ্ড, কোনো সরকারি সনদ নয়। নিজের কাজ মূল্যায়নের একটি সাহায্যকারী:

`rubric.mjs` · JavaScript — চলবে: Node.js

```
// ১০০ নম্বরের মূল্যায়ন: প্রতিটি ক্ষেত্রে নিজের নম্বর দিন, তারপর ব্যাখ্যা।
```

```
const rubric = [
  ['Requirement ও User Flow', 10],
  ['JavaScript-এর নকশা', 15],
  ['API ও Authentication', 15],
  ['Data-র সামঞ্জস্য', 15],
  ['Test ও Debugging', 15],
  ['নিরাপত্তা ও গোপনীয়তা', 10],
  ['Deployment ও Monitoring-এর পরিকল্পনা', 10],
  ['Documentation ও যোগাযোগ', 10],
];

function assess(scores) {
  const total = rubric.reduce((sum, [area, max]) => {
    const given = scores[area] ?? 0;
    if (given < 0 || given > max) throw new RangeError(`${area}: 0-${max}`);
    return sum + given;
  }, 0);
  const band = total >= 85 ? 'শক্তিশালী Capstone'
    : total >= 70 ? 'ভালো ভিত্তি'
      : total >= 50 ? 'নির্দিষ্ট উন্নতি দরকার' : 'Requirement ও Test আগে';
  const weakest = rubric.map(([area, max]) => [area, (scores[area] ?? 0) / max])
    .sort((a, b) => a[1] - b[1])[0][0];
  return { total, band, weakest };
}

console.log(assess({ 'Requirement ও User Flow': 9, 'JavaScript-এর নকশা': 13,
  'API ও Authentication': 13, 'Data-র সামঞ্জস্য': 14, 'Test ও Debugging': 13,
  'নিরাপত্তা ও গোপনীয়তা': 7, 'Deployment ও Monitoring-এর পরিকল্পনা': 5,
  'Documentation ও যোগাযোগ': 8 }));
```

Output

```
{
  total: 82,
  band: 'ভালো ভিত্তি',
  weakest: 'Deployment ও Monitoring-এর পরিকল্পনা'
}
```

সবচেয়ে দুর্বল ক্ষেত্রটি নম্বরের অনুপাতে বের করা হয় (৫/১০ = ৫০%), মোট নম্বরে নয়—উন্নতির পরের পদক্ষেপ সেখান থেকেই।

৩০.১৪ পেশাদার Roadmap: ১২ সপ্তাহ

সপ্তাহ	মনোযোগ	যা তৈরি করবেন
১-২	ভিত্তি পুনরালোচনা, Async, Git	Test-সহ একটি Utility Library

সপ্তাহ	মনোযোগ	যা তৈরি করবেন
৩-৪	DOM, Accessibility, API ডাক	Responsive Package Catalog
৫-৬	Node.js, Auth, REST	Booking API
৭-৮	SQL, Transaction, ORM	টেকসই Booking প্রবাহ
৯-১০	Test, TypeScript, নিরাপত্তা	CI-তে পরীক্ষিত Refactor
১১	Docker, Deploy, Monitoring	Staging Demo
১২	Portfolio, README, Interview	Case Study ও Walkthrough

Portfolio: শুধু Screenshot নয়। প্রতিটি Project-এর জন্য লিখুন: সমস্যা কী ছিল; Architecture ও কেন; কোন Trade-off নিলেন; কীভাবে প্রমাণ করলেন (Test, Demo); কী সীমাবদ্ধতা আছে; এবং আবার করলে কী অন্যভাবে করতেন। TravelMaster Pro-র README-র "Requirement থেকে প্রমাণ" ও "যা নেই" অংশ এর একটি নমুনা। Git-এর ইতিহাস পরিচ্ছন্ন রাখুন—ছোট, অর্থপূর্ণ Commit।

Interview-এর প্রস্তুতি: Event Loop ও Microtask; Closure ও Scope; Async Error সামলানো; Authentication বনাম Authorization; SQL Transaction ও Race Condition; Caching ও তার Invalidation; Idempotency; এবং একটি ছোট System Design ("একটি Booking ব্যবস্থা নকশা করুন")। প্রতিটি উত্তরে একটি নিজের Code-এর উদাহরণ দিতে পারলে—"আমার Capstone-এ এই Race Condition এভাবে ধরেছিলাম"—তা যেকোনো মুখস্থ সংজ্ঞার চেয়ে শক্তিশালী।

শেখা চালিয়ে যাওয়া: প্রতিটি নতুন প্রযুক্তি শেখার আগে নিজেকে জিজ্ঞাসা করুন: এটি কোন সমস্যার সমাধান করে? এর মূল্য কী? এটি ছাড়া কীভাবে করতাম? আর একটি Open Source Project-এ ছোট অবদান—একটি Documentation সংশোধন বা একটি Bug-এর Test—আসল দলের সঙ্গে কাজের অভ্যাস তৈরি করে।

৩০.১৫ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Client-এর পাঠানো দাম বিশ্বাস	যে কেউ ১ টাকায় Booking	দাম সব সময় Server-এর Catalog থেকে
`typeof` দিয়ে সংখ্যা যাচাই	NaN, Infinity, ভগ্নাংশ পার হয়	`Number.isInteger` ও সীমা
পড়ে, তারপর লেখা (Inventory)	একসঙ্গে অনুরোধে Overbooking	শর্তযুক্ত UPDATE, Transaction, CHECK
Password সাধারণ লেখায় বা দ্রুত Hash-এ	ফাঁস হলে সব Account বিপদে	scrypt বা Argon2id, প্রতিজনের Salt
Token সরাসরি Database-এ	Database ফাঁস মানে Session চুরি	Token-এর Hash

ভুল	কেন সমস্যা	সমাধান
অন্যের Booking-এ 403	কোন ID আছে তা ফাঁস	404, এবং Query-তেই মালিকানা
Double-submit শুধু Button নিষ্ক্রিয় করে ঠেকানো	Network Retry-তে তবু দুটি	Idempotency Key, Server-এ UNIQUE
`innerHTML` দিয়ে Server-এর লেখা বসানো	XSS	`textContent`
Token `localStorage`-এ	XSS Script পড়ে নেয়	Memory বা HttpOnly Cookie
"Test পাস, তাই Production-প্রস্তুত"	নিরাপত্তা, Scale, Operations অপরীক্ষিত	সৎ সীমাবদ্ধতার তালিকা ও পরিকল্পনা

৩০.১৬ Debugging নির্দেশিকা

১. **Response-এর `error.code` আগে:** VALIDATION হলে `details`; UNAUTHORIZED হলে Token ও মেয়াদ; KEY_CONFLICT হলে একই Key-তে ভিন্ন Body।

২. **`audit_events` Table:** কে কখন Login করল, কোন Booking তৈরি বা বাতিল হলো—সময়ের ক্রমে।

৩. **আসনের হিসাব মিলানো:** `seats` + CONFIRMED Booking-এর যাত্রীর যোগফল = প্রাথমিক আসন। না মিললে কোথাও Transaction-এর বাইরে লেখা হয়েছে।

৪. **Browser-এর DevTools:** Network Tab-এ Request ও Response; Console-এ CSP লঙ্ঘন।

৫. **Domain আলাদা করে:** Bug HTTP-তে না Domain-এ? `service` সরাসরি ডেকে দেখুন—Test-এর `domain()` Helper-এর মতো।

৬. **সময়-নির্ভর Bug:** ঘড়ি Inject করা (`now`), তাই মেয়াদ বা Rate Limit-এর Bug সঙ্গে সঙ্গে পুনরুৎপাদন করা যায়।

৩০.১৭ পেশাদার অভ্যাস

- **প্রতিটি Requirement একটি Test-এ প্রমাণিত, প্রতিটি Test একটি Requirement-এর জন্য।**
- **Trust Boundary-তে যাচাই, ভেতরে Type-এর নিশ্চয়তা।**
- **Database-এর Constraint শেষ পাহারাদার।**
- **নিরাপত্তার ডিফল্ট কঠোর:** CSP, `no-store`, সাধারণ Error-বার্তা, ন্যূনতম তথ্য।
- **সিদ্ধান্ত ADR-এ, সীমাবদ্ধতা README-তে।**
- **যা করা হয়নি তা স্পষ্ট বলুন**—সৎ "বাস্তবায়িত নয়" একটি মিথ্যা "সম্পূর্ণ"-এর চেয়ে অনেক বেশি পেশাদার।

৩০.১৮ অনুশীলনী

প্রাথমিক স্তর

১. `TEST@EXAMPLE.COM` দিয়ে নিবন্ধনের পর `test@example.com` দিয়ে আবার নিবন্ধন করলে কী হয় এবং কেন?

২. Client Booking-এর Body-তে `{ "price": 1 }` পাঠালে মোট দাম কত হয়? কোন Line এটি নিশ্চিত করে?

৩. একই Idempotency Key ও একই Body দুবার পাঠালে কী হয়: Status, Booking ID, আসন?

৪. Token ছাড়া `GET /api/bookings` পাঠালে Response কী, এবং কেন Body পড়ার আগেই তা ঘটে?

মধ্যম স্তর

৫. একই Key-তে `travelers` বদলে পাঠালে 409 কেন, নীরবে আগের ফল নয় কেন?

৬. ব্যবহারকারী A-র Booking ব্যবহারকারী B বাতিল করতে চাইলে 404 কেন, 403 নয় কেন?

৭. একই Booking দুবার বাতিল করলে আসন কতবার বাড়ে? Code-এর কোন অংশ তা নিশ্চিত করে?

৮. Test লিখুন: মেয়াদের এক Millisecond আগে Token বৈধ, ঠিক মেয়াদে অবৈধ; এবং মেয়াদোত্তীর্ণ Token-এ Booking হয় না।

চ্যালেঞ্জ স্তর

৯. Schema-র প্রতিটি Constraint কোন Requirement রক্ষা করে? একটি নতুন Constraint প্রস্তাব করুন।

১০. Production-এ যাওয়ার আগে অগ্রাধিকার অনুযায়ী পাঁচটি কাজ লিখুন এবং কেন এই ক্রম।

১১. Catalog-এ দামের সীমা, শুধু খালি আসনের Package এবং Keyset Pagination যোগ করুন।

১২. Payment-এর জন্য "Reservation Expiry" নকশা করুন: Booking PENDING_PAYMENT অবস্থায় ১৫ মিনিট আসন ধরে রাখবে, তারপর ছেড়ে দেবে।

৩০.১৯ অনুশীলনীর সমাধান

সমাধান ১ — দ্বিতীয় নিবন্ধন 409 EMAIL_TAKEN পায়। `validateRegistration` Email-কে `trim().toLowerCase()` করে, তাই দুটিই `test@example.com` হিসেবে Database-এ যায়, আর `users.email`-এর UNIQUE Constraint দ্বিতীয়টি প্রত্যাখ্যান করে। `register` সেই Constraint-এর Error ধরে বোধগম্য বার্তা দেয়। স্বাভাবিক না করলে একই মানুষ দুটি Account পেতেন, আর Login-এ কোন Account—তা বড়-ছোট হাতের অক্ষরের ওপর নির্ভর করত (`auth.test.mjs`-এর দ্বিতীয় Test)।

সমাধান ২ — ২৫,০০০ টাকা (BT-4D, ২ জন × ১২,৫০০)। `validateBooking` শুধু `packageId`, `travelers` ও `idempotencyKey` ফেরত দেয়— `price` Field Domain-এ পৌঁছায়ই না। আর `createBooking`-এ মোট দাম `pkg.price_bdt * travelers`, যেখানে `pkg` Database-এর Catalog থেকে পড়া। Demo-র প্রথম Line এবং `service.test.mjs`-এর প্রথম Test এটি প্রমাণ করে।

সমাধান ৩ — প্রথমবার 201, দ্বিতীয়বার 200; Booking ID একই; আসন একবারই কমে। Transaction-এর শুরুতে `(user_id, idempotency_key)` দিয়ে খোঁজা হয়, পাওয়া গেলে অনুরোধ মিলিয়ে আগের Booking `replayed: true` সহ ফেরত, এবং HTTP স্তর তা দেখে 200 দেয়। Database-এর UNIQUE Constraint নিশ্চিত করে যে দুটি একসঙ্গে আসা অনুরোধেও দুটি সারি হবে না।

সমাধান ৪ — 401, Body: `{ "error": { "code": "UNAUTHORIZED", "message": "Login প্রয়োজন" } }`। Route Handler-এ প্রথম কাজ `user(req)`, যা `service.authenticate` ডাকে; Token না থাকলে সেখানেই AppError। Body পড়ার আগে Authentication মানে পরিচয়হীন কেউ বড় Body পাঠিয়ে Server-এর সময় ও Memory নষ্ট করতে পারে না।

সমাধান ৫ — একই Key মানে Client দাবি করছে "এটি সেই আগের অনুরোধ", অথচ বিষয়বস্তু ভিন্ন—এটি Client-এর Bug (যেমন প্রতিটি Retry-তে নতুন Form-মান কিন্তু পুরোনো Key)। নীরবে আগের ফল দিলে ব্যবহারকারী ভাববেন তিনজনের Booking হয়েছে, আসলে হয়েছে দুজনের। 409 KEY_CONFLICT স্পষ্টভাবে অমিল জানায়। TravelMaster-এর Frontend Form-এ কিছু বদলালেই `pendingKey` মুছে দেয়, তাই সাধারণ ব্যবহারে এটি ঘটে না।

সমাধান ৬ — 403 বলত "এই Booking আছে, কিন্তু আপনার নয়"। তাহলে কেউ ID ১, ২, ৩... দিয়ে চেষ্টা করে জানতে পারত কতগুলো Booking আছে, কোনগুলো সক্রিয়—ব্যবসার তথ্য ও গোপনীয়তা ফাঁস (Enumeration)। 404 বলে "আপনার জন্য এমন কিছু নেই", যা সত্য। Query-তে `WHERE id = ? AND user_id = ?` থাকায় অন্যের Booking Domain-এর চোখে অস্তিত্বহীন।

সমাধান ৭ — একবার। `cancelBooking` Transaction-এ সারি পড়ে; `status` আগে থেকেই CANCELLED হলে কোনো UPDATE না করে একই ফল ফেরত দেয়। আসন ফেরত দেওয়া ও অবস্থা বদলানো একই Transaction-এ, তাই দুটি একসঙ্গে আসা বাতিল-অনুরোধেও দ্বিতীয়টি প্রথমটির Commit-এর পরে CANCELLED দেখবে (`BEGIN IMMEDIATE`-এর Lock)।

সমাধান ৮

token-expiry.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { domain } from '../../project/travel-master-pro/tests/helpers.mjs';
```

```
test('মেয়াদের এক মুহূর্ত আগে বৈধ, ঠিক মেয়াদে অবৈধ', () => {
  const { service, user, clock } = domain({ tokenTtlMs: 30 * 60_000 });
  const { token } = user('Rafi');
  clock.now += 30 * 60_000 - 1;
  assert.equal(service.authenticate(token).name, 'Rafi');
  clock.now += 1;
  assert.throws(() => service.authenticate(token), { code: 'UNAUTHORIZED' });
});

test('মেয়াদ শেষে Booking-এর চেষ্টাও 401-এর পথে', () => {
  const { service, user, clock } = domain({ tokenTtlMs: 1000 });
  const { token } = user('Mina');
  clock.now += 5000;
  assert.throws(() => {
    const me = service.authenticate(token); // HTTP স্তর আগে এটিই ডাকে
    service.createBooking(me.id, { packageId: 'BT-4D', travelers: 1,
      idempotencyKey: 'late-key-0001' });
  }, { code: 'UNAUTHORIZED' });
});
```

Output

```
ok 1 - মেয়াদের এক মুহূর্ত আগে বৈধ, ঠিক মেয়াদে অবৈধ
ok 2 - মেয়াদ শেষে Booking-এর চেষ্টাও 401-এর পথে
# tests 2
# pass 2
# fail 0
```

ঘড়ি Inject করা বলে Test-এ ৩০ মিনিট অপেক্ষা লাগে না। সীমার দুই পাশে (এক Millisecond আগে ও ঠিক মেয়াদে) পরীক্ষা করা Boundary Testing-এর মূল নিয়ম— `<` বনাম `<=` ধরনের ভুল এখানেই ধরা পড়ে।

সমাধান ৯ — `users.email UNIQUE`: এক Email এক Account (FR2)। `sessions.token_hash PRIMARY KEY` ও `expires_at`: স্বল্পমেয়াদি Session (FR3)। `packages.price_bdt CHECK (> 0)`: শূন্য বা ঋণাত্মক দামে বিক্রি নয় (FR4)। `packages.seats CHECK (>= 0)`: Overbooking নয় (FR5, Concurrency)। `bookings.user_id`, `package_id` FOREIGN KEY: অস্তিত্বহীন ব্যবহারকারী বা Package-এ Booking নয়। `travelers CHECK (BETWEEN 1 AND 10)`: Validation-এর দ্বিতীয় স্তর। `status CHECK (IN ...)`: অজানা অবস্থা নয়। `UNIQUE (user_id, idempotency_key)`: Idempotency (FR5)। প্রস্তাব: `CHECK (total_bdt % travelers = 0)` নয় (দাম বদলালে ভাঙবে); বরং একটি `price_at_booking` Column, যাতে পরে Catalog-এর দাম বদলালেও Booking-এর দাম ইতিহাসে ঠিক থাকে।

সমাধান ১০ — (১) **Payment** (PENDING_PAYMENT, Webhook যাচাই, Expiry): এটি ছাড়া ব্যবসাই চলে না। (২) **HTTPS ও Cookie-ভিত্তিক Session, CSRF:** Token Network-এ খোলা থাকলে বাকি সব নিরাপত্তা অর্থহীন। (৩) **PostgreSQL ও Backup/Restore Drill:** টেকসই Data এবং হারালে ফেরানোর প্রমাণিত পথ। (৪) **ভাগ-করা Rate Limit ও Monitoring (Log, Metrics, Alert):** আক্রমণ ও ব্যর্থতা দেখতে পাওয়া। (৫) **CI ও ধাপে ধাপে Deploy, Rollback:** প্রতিটি পরিবর্তন নিরাপদে। ক্রমের যুক্তি: আগে যা ছাড়া সেবা অর্থহীন বা বিপজ্জনক, তারপর যা ব্যর্থতা থেকে রক্ষা করে, তারপর যা পরিবর্তনকে নিরাপদ করে।

সমাধান ১১

catalog-filter.mjs · JavaScript — চলবে: Node.js

```
import { openDatabase } from '../../project/travel-master-pro/src/db.mjs';

// Catalog-এ দামের সীমা, শুধু খালি আসনের Package, এবং Keyset Pagination।
// সব মান Parameter-এ (?), তাই SQL Injection-এর সুযোগ নেই।
function listPackages(db, { maxPrice = Number.MAX_SAFE_INTEGER,
  onlyAvailable = false, limit = 2, after = '' } = {}) {
  if (!Number.isInteger(limit) || limit < 1 || limit > 50) {
    throw new RangeError('limit: ১-৫০');
  }
  const rows = db.prepare(`SELECT id, price_bdt AS price, seats FROM packages
    WHERE price_bdt <= ? AND (? = 0 OR seats > 0) AND id > ?
    ORDER BY id LIMIT ?`)
    .all(maxPrice, onlyAvailable ? 1 : 0, after, limit + 1)
    .map((row) => ({ ...row }));
  const page = rows.slice(0, limit);
  return { page, next: rows.length > limit ? page.at(-1).id : null };
}

const db = openDatabase();
db.exec("UPDATE packages SET seats = 0 WHERE id = 'SYL-2D'");
let cursor = '';
do {
  const { page, next } = listPackages(db, { maxPrice: 13_000,
    onlyAvailable: true, limit: 1, after: cursor });
  console.log(page.map((p) => `${p.id} ${p.price}`).join(' | '),
    next ? `(পরের পাতা: ${next}-এর পর)` : '(শেষ)');
  cursor = next;
} while (cursor);
```

Output

```
BT-4D 12500 (পরের পাতা: BT-4D-এর পর)
CXB-3D 9800 (শেষ)
```

একটি বেশি সারি (`limit + 1`) চেয়ে জানা যায় পরের পাতা আছে কি না, আলাদা COUNT Query ছাড়াই। Keyset (`id > ?`) OFFSET-এর মতো গভীর পাতায় ধীর হয় না (অধ্যায় ২৬)। API-তে এটি `GET /api/packages?maxPrice=13000&available=1&after=BT-4D` হবে, আর Query String-এর প্রতিটি মান একই কঠোরতায় যাচাই করতে হবে।

সমাধান ১২ — Booking তৈরিতে অবস্থা `PENDING_PAYMENT` ও `hold_expires_at = now + 15 মিনিট`; আসন তখনই কমে (শর্তযুক্ত UPDATE, আগের মতো)। Payment Gateway-এর Webhook এলে (Signature যাচাই, Event ID-তে Inbox) একটি Transaction-এ `PENDING_PAYMENT` থেকে `CONFIRMED`—শুধু যদি তখনো PENDING থাকে। একটি Background Job প্রতি মিনিটে `WHERE status = 'PENDING_PAYMENT' AND hold_expires_at < now` খুঁজে প্রতিটিকে Transaction-এ `EXPIRED` করে আসন ফেরত দেয়। সূক্ষ্ম ক্ষেত্র: মেয়াদ শেষের পরে Payment-এর Webhook এলে—আসন হয়তো অন্যকে বিক্রি হয়েছে—তাই স্বয়ংক্রিয় Refund ও গ্রাহককে জানানো। Status-এর CHECK-এ নতুন দুটি মান যোগ হবে (একটি Migration)।

৩০.২০ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Booking-এর দাম Server-এ হিসাব করার প্রধান কারণ?

ক) UI দ্রুত হয়

খ) Client-এর পাঠানো মান বদলানো যায়, তাই বিশ্বাসযোগ্য নয়

গ) CSS সহজ হয়

ঘ) Cache ভালো কাজ করে

প্রশ্ন ২. Idempotency Key-এর উদ্দেশ্য?

ক) একই অনুরোধ বারবার এলেও কাজ একবারই

খ) SEO উন্নতি

গ) ছবি ছোট করা

ঘ) Login দ্রুত করা

প্রশ্ন ৩. Authentication ও Authorization-এর পার্থক্য?

ক) দুটি একই

খ) Network বনাম Disk

গ) পরিচয় যাচাই বনাম অনুমতি যাচাই

ঘ) HTML বনাম CSS

প্রশ্ন ৪. একসঙ্গে আসা অনুরোধে Overbooking ঠেকাতে কী দরকার?

ক) `setTimeout`

খ) `localStorage`

গ) একটি `alert`

ঘ) শর্তযুক্ত UPDATE বা Lock-সহ Database Transaction

প্রশ্ন ৫. TravelMaster-এ মেয়াদোত্তীর্ণ Token-এ কী উত্তর আসে?

ক) 401

খ) 404

গ) 409

ঘ) 200

প্রশ্ন ৬. অন্যের Booking বাতিলের চেষ্টায় TravelMaster 404 দেয় কেন?

ক) Server-এর Bug

খ) Booking-এর অস্তিত্বের তথ্যও ফাঁস না করতে

গ) 403 HTTP-তে নেই

ঘ) Booking মুছে ফেলা হয়েছে

প্রশ্ন ৭. Database-এ Token-এর বদলে তার Hash রাখার সুবিধা?

ক) Query দ্রুত

খ) Database ছোট

গ) Database ফাঁস হলেও চুরি করা মান দিয়ে Login করা যায় না

ঘ) Token-এর মেয়াদ বাড়ে

প্রশ্ন ৮. ADR কী লিখে রাখে?

ক) Architecture-এর সিদ্ধান্ত, বিকল্প ও তার মূল্য

খ) Animation

গ) শুধু CSS

ঘ) Invoice

প্রশ্ন ৯. Frontend-এ Server-এর লেখা DOM-এ বসাতে কোনটি নিরাপদ?

ক) `innerHTML`

খ) `document.write`

গ) `eval`

ঘ) `textContent`

> **প্রশ্ন ১০.** Portfolio-র Case Study-তে সবচেয়ে জরুরি কী?
>
> ক) শুধু Screenshot
>
> খ) শুধু রং
>
> গ) সমস্যা, সিদ্ধান্ত, প্রমাণ ও সীমাবদ্ধতা
>
> ঘ) শুধু শিরোনাম

৩০.২১ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ক · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Demo-তে Client `price: 1` পাঠালেও ২৫,০০০ টাকা।
২	ক	একই Key-তে দ্বিতীয় অনুরোধ আগের Booking ফেরত দেয়, আসন কমে না।
৩	গ	Login পরিচয় দেয়; মালিকানা যাচাই অনুমতি ঠিক করে।
৪	ঘ	উদাহরণ ০২ ও Concurrency Test: চার Thread-এও ঠিক ২০টি।
৫	ক	`authenticate`-এ `expires_at <= now()` হলে UNAUTHORIZED।
৬	খ	403 বললে ID অনুমান করে কোন Booking আছে তা জানা যেত।
৭	গ	Hash থেকে আসল Token ফেরানো যায় না।
৮	ক	যেমন ADR-001: কেন SQLite, কোন বিকল্প, কী মূল্য।
৯	ঘ	`textContent` লেখাকে লেখা হিসেবেই রাখে, Code হিসেবে চালায় না।
১০	গ	নিয়োগকর্তা দেখতে চান আপনি কীভাবে ভাবেন ও প্রমাণ করেন।

৩০.২২ অধ্যায়ের সারসংক্ষেপ

- Mastery মানে Correctness, নিরাপত্তা, রক্ষণযোগ্যতা ও পর্যবেক্ষণ মিলিয়ে সমস্যা সমাধান, এবং সিদ্ধান্তের ব্যাখ্যা।
- Requirement থেকে API চুক্তি, প্রতিটি Requirement একটি Code ও একটি Test-এ প্রমাণিত।
- Trust Boundary-তে কঠোর যাচাই; `typeof` যথেষ্ট নয়।
- scrypt ও Salt, সমান-সময়ের তুলনা, একই Error-বার্তা, Token-এর Hash, মেয়াদ ও Rate Limit।
- দাম Server-এর; Idempotency Key ও UNIQUE; শর্তযুক্ত UPDATE ও Transaction; Constraint শেষ পাহারাদার।
- Domain ও Transport আলাদা; একটি Error চুক্তি; CSP ও `no-store`।
- Frontend: `textContent`, Memory-তে Token, দুই স্তরের Double-submit সুরক্ষা, Accessibility-র ভিত্তি।

- Test-এর পিরামিড: Unit থেকে Concurrency, Type Check ও আসল Browser।
- ADR, সৎ সীমাবদ্ধতা, Production-এর পরিকল্পনা, ১০০ নম্বরের মূল্যায়ন ও ১২ সপ্তাহের Roadmap।

৩০.২৩ বই শেষ, শেখা নয়

এই ত্রিশটি অধ্যায়ের লক্ষ্য ছিল আপনাকে নিজে Engineering-এর সিদ্ধান্ত নিতে সক্ষম করা। আপনি একটি নাম দিয়ে শুরু করেছিলেন—একটি Variable, একটি অভিবাদন—আর শেষ করলেন এমন একটি ব্যবস্থায়, যেখানে চারটি Thread একসঙ্গে শেষ আসন চাইলেও হিসাব মেলে। পথে শিখেছেন যে ভালো Code শুধু কাজ করে না; সে জানায় কখন কাজ করছে না, কেন, এবং কীভাবে ঠিক করতে হবে।

প্রযুক্তি বদলাবে: নতুন Framework, নতুন Runtime, নতুন হাতিয়ার আসবে। কিন্তু এই বইয়ের মূল প্রশ্নগুলো বদলাবে না—Input কি বিশ্বাসযোগ্য? একসঙ্গে দুজন এলে কী হবে? ব্যর্থ হলে কে জানবে? কীভাবে প্রমাণ করব? এই প্রশ্নগুলো যেকোনো নতুন প্রযুক্তিতে নিয়ে যান। শুভকামনা।

৩০.২৪ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Capstone	শেখা সব দক্ষতা একত্র করা চূড়ান্ত Project।
Trust Boundary	যেখানে বাইরের অবিশ্বস্ত তথ্য ব্যবস্থায় ঢোকে।
Traceability	Requirement থেকে Code ও Test পর্যন্ত যোগসূত্র।
Salt	প্রতিটি Password-এর জন্য আলাদা এলোমেলো মান।
Timing-safe Comparison	তুলনার সময় দেখে কিছু বোঝা যায় না এমন তুলনা।
Session Token	Login-এর পরে পরিচয় বহনকারী গোপন মান।
Enumeration	ID অনুমান করে কী আছে তা খুঁজে বের করা।
Conditional Update	যাচাই ও পরিবর্তন একই Statement-এ।
Content-Security-Policy	কোন উৎসের Script বা Style চলবে, তার নিয়ম।
Composition Root	যেখানে সব স্তর জোড়া লাগানো হয়।
ADR	Architecture-এর সিদ্ধান্তের লিখিত নথি।
Quality Gate	Release-এর আগে যে যাচাইগুলো পার হতে হয়।

৩০.২৫ আরও পড়ুন

- **OWASP — Password Storage Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html` — scrypt, Argon2id ও Parameter।
- **OWASP — Session Management Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html` — Token, Cookie ও মেয়াদ।
- **OWASP — Authorization Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html` — মালিকানা যাচাই ও IDOR।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP` — CSP-র প্রতিটি Directive।
- **Node.js — SQLite:** `https://nodejs.org/api/sqlite.html` — `node:sqlite`-এর API।
- **Michael Nygard — Documenting Architecture Decisions:** `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions` — ADR-এর মূল প্রস্তাব।
- **web.dev — Learn Accessibility:** `https://web.dev/learn/accessibility` — Accessible UI-র ভিত্তি।